A COVER-FREE BIT SUPPLIER FOR INTEGER MULTIPLICATION BELOW $n \log n$ IN LEAN

DAN BROWN* AND KENNY DANIEL

Abstract. The Lean development [5] proves that one fixed machine of Mathlib's `Turing.FinTM2` model multiplies n-bit numbers in time $O\big(n(\log n)^{1-\kappa}\big)$. With the T16 complex supplier in place, its bit interchange, the Shared50 network of [6] with exponent $\tau = 1-296\cdot10^{-11}$, bounds the saving at $\kappa = 88279\cdot2^{-46} \approx 2^{-29.57}$. We replace that interchange with the two-stage partial-swap word of PR97 [7], built on Swapnil Jain's round-7 witness [8]. The word needs no routes, charts, cover classes or helper hosting, so it runs on the existing recursive interchange machine. The proof adds a constructive exchange lemma, which makes a rank-r frame change r child calls between lower-triangular passes; an explicit height bound, which fixes the machine's prime; a machine generic in the bit network; and a seam that selects the supplier. The new saving is $\kappa = 111364923 \cdot 2^{-46} \approx 2^{-19.27}$. We also record why the selected PR589 supplier does not port, rank the cover-free suppliers of [1], and describe the next stage, the opposite-bank compiler of [4], at about $2^{-14.5}$.

1. Introduction

The saving κ of [5] is the least of five margins of the compact multiplier, with $\varepsilon = 31/64$ and chunk exponent $c = 7/8$:

$$\varepsilon c(1-\tau), \quad \varepsilon(1-\lambda'), \quad (1-\varepsilon)(1-\tau), \quad \tfrac{1}{2} - \delta - \varepsilon(1+2^{-33}), \quad \varepsilon(1-c).$$

Here τ is the exponent of the bit interchange, $\lambda < \lambda' < 1$ are the layer exponents, and the constraints $\tau < \lambda$ and $\sigma < \lambda$ tie the layers to both suppliers. Kenny Daniel's T16 node on the layer machine gives $\sigma < 999/1000$ (`br_branching_999`). With T16 the binding margin is the layout margin $\varepsilon c(1-\tau)$, so the bit supplier sets κ: at Shared50's $1-\tau = 2.96\cdot10^{-9}$ this is $88279\cdot2^{-46}$.

Each claim carries a tag. [Lean] marks a theorem checked in [5] under the name given, on the axioms `propext`, `Classical.choice` and `Quot.sound`. [paper] marks a written argument. [computation] marks a number computed by a script from pinned upstream data.

2. The machine model

A node of the recursive interchange machine holds records addressed by two fields H and D of m sub-fields each, and interchanges them. For an idempotent U write

$$D_U(h,d) = \big((1-U)h + Ud,\ Uh + (1-U)d\big),$$

so the parent interchange is D_1. The node's cheap operations are the field operations of `AffineFieldProgram.Op`: scalings, ordered shifts, the cross shears $D \mathrel{+}= KH$, and XOR gates between role arrays. On the address coordinates of one role these act by lower triangular matrices in the H-before-D order. A child is `interchange i j`, which swaps H_i with D_j for independent i and j, on volume V/W for W roles. A node with s children over W roles costs $O(V)$ plus its children, and the uniform recursion gives $O\big(Vw^\tau\big)$ for every τ with $s/W < m^\tau$.

Date: October 2026.

*Author of *The Da Vinci Code*.

3. Choosing a bit supplier

3.1. **The Shared50 family.** [computation] Shared50 has $h = 50$, $m = h^3$, and deficit $\eta = D/(Wm) = 23/661055000$. With rank-one children its saving is $\eta/\ln m = 2.96 \cdot 10^{-9}$. In the three-stage cube family $\eta = (v - 6h^2)/(2(v + R + h)h^3)$ with $v = \binom{h}{3}$; more roles R only lower it, and the loss $6h^2$ forces $h > 36$. The family is capped at $7.99 \cdot 10^{-8}$ with rank-one children. A better shear program gains nothing, because lower-triangular factors preserve the rank of the H-to-D block, so `exists_prime_shear`'s rank A children are already minimal.

3.2. **PR589.** The selected bit supplier of [1], PR589, has $a_b = 10/5769$. Our earlier note [3] shows that its accounting assumes free port routes, which this machine has to pay for. A further study [paper][computation] sharpens that verdict:

- In upstream's geometry, 970 of the word's 5472 gates join registers of two different ports, so every column needs common coordinates at every invocation.
- With split windows the route question reduces to one scalar condition per port. One window family serves 88 of the 111 E8 ports with no route cost, with full-size solves to residuals below 10^{-8}.
- Hosting the full word over those 88 ports costs about 0.66 rank units per role, so the moment at $\lambda = 1$ is 1.0073 and nothing is saved. Keeping 97 ports with corrections gives $a_b \le 4.06 \cdot 10^{-4}$ only if a corner defect can be repaired with two rank-one children per family, the parity minimum; no such repair is known.
- A word whose ports all share one line saves nothing in any geometry.

3.3. **Cover-free suppliers.** [computation] Every supplier of [1] with $a_b > 1.24 \cdot 10^{-4}$ needs a class-indexed cover or the compressed nine-stage geometry with port routes. Those that run on this machine:

supplier	mechanism	a_b
#104 on PR97	opposite-bank whole projector, design-B frames	$1.2402 \cdot 10^{-4}$
#104 on PR97	opposite-bank whole projector, plain frames	$9.70 \cdot 10^{-5}$
PR97	flag-basis corner runs	$6.3974 \cdot 10^{-5}$
PR97	rank-one children	$3.734 \cdot 10^{-6}$
cube family	rank-one ceiling	$\le 7.99 \cdot 10^{-8}$
Shared50	rank-one children	$2.96 \cdot 10^{-9}$

This note proves the PR97 row with rank-one children.

4. Frame changes on the machine

Theorem 4.1 (`M8.exchange_factor`, `M8.ExchangeProgram.program_run`, `program_interchanges`; [Lean])**.** *Let $U = E_1 \Pi E_2$ be an idempotent over a commutative ring, with E_1, E_2 invertible lower triangular and Π a pivot matrix with pivot pairs L. Put $M = E_2 E_1$, $P_E = \Pi\Pi^{\mathsf{T}}$, $P_F = \Pi^{\mathsf{T}}\Pi$ and*

$$K = MP_E - P_F M + P_F M P_E - \Pi^{\mathsf{T}}.$$

Then D_U equals: transform H by E_1^{-1} and D by E_2; shear D += KH; swap H_i with D_j for each $(i, j) \in L$; shear D −= KH; transform back. It uses exactly $|L| = \operatorname{rank} U$ interchanges.

Sketch. In the coordinates $(E_1^{-1}h, E_2 d)$ the map is $g = 1 + CR$ with $C = [\Pi; -M\Pi]$ and $R = [-M, 1]$. Idempotency of U is $\Pi M \Pi = \Pi$. The matrix K satisfies $KE = ME - F$ and $F^{\mathsf{T}}K = E^{\mathsf{T}} - F^{\mathsf{T}}M$ for the pivot columns $\Pi = EF^{\mathsf{T}}$, so conjugating by the shear $S = \left[\begin{smallmatrix} 1 & 0 \\ K & 1 \end{smallmatrix}\right]$ gives $SgS^{-1} = 1 + [E; -F][-E^{\mathsf{T}}, F^{\mathsf{T}}]$, the swap on the pivot pairs. □

For nested frames $A \le B$ the frame change is $D_A D_B = D_{B-A}$ (`exchange_comp`), of rank rank B−rank A (`rank_sub_add`) [Lean]. With rank-one children the cost along a wire telescopes to its final rank minus its initial rank, so the count s does not depend on the intermediate frames [paper].

4.1. **The prime.** The machine runs over $\mathbb{Z}/q^b$ for one odd prime q. Lower-triangular factorizations are not unique, so a factorization chosen by `Classical.choice` has no height. The development proves a height-bounded elimination (`M8.Height.exists_bounded`) [Lean]: for an $n \times n$ rational matrix whose entries have numerator and denominator at most H, some factorization has all entries below an explicit $G(n, H)$. Every number the exchange program needs is then below `coefficientBound` $= 2^e$, e a sealed choice above $2^{2^{16}}$, for $n \le 529$ and $H = 10^{10000}$ (`numbers_le_coefficientBound`) [Lean]. The kernel never evaluates the tower.

Two edits to the Shared50 network files make that prime available. `Shared50ModularControl.prime` is chosen beyond `coefficientBound` and Shared50's denominators (`coefficientBound_lt_prime`), and the coefficient classes `Occurs` and `ScaleOccurs` admit every rational within the bound [Lean].

5. The PR97 word

[computation] The word has $h = 23$, $v = \binom{23}{3} = 1771$, $m = h^2 = 529$, $N = v^2 = 3136441$, $R = 28866$ side roles per invocation, $W = 2N + 2vR = 108516254$ roles, and rank mass

$$s = Wm - N + 2vh(h-1) = 57403754177.$$

The pinned ledger replays: 1080616 events, s as stated, and the complete forward and reflected $\mathbb{F}_2$ shear.

5.1. **Plain frames.** The development uses the simplest nested frames the gates allow [paper][computation]. Readouts start at frame 0; the source of triple S sits at the line t_S; node n of the side DAG sits at $\mathrm{span}(n) = \sum_{S \in \mathrm{supp}\, n} \mathbb{Q} t_S$; the roots read at the hyperplanes $t_T^{\perp}$ and the star spaces U_c. The frames are the G-orthogonal projectors for $G = I - J/9$ on $\mathbb{Q}^{23}$ (`M8.Frames.Projector`). Nondegeneracy is a theorem: every support lies in a star, and on U_c the form G is the Euclidean form of the pair vectors, which is positive definite. The plain word has the same s as upstream's design-B frames. Every projector entry has numerator and denominator at most 10^{200} (`frame_height_le`) [Lean].

5.2. **The local word.** The local word is OPS' ; RR ; OPS'^{-1} ; OPS ; RR ; OPS^{-1}, with OPS the compiled XOR list of the side DAG and RR the root reads [Lean]:

- `compile_correct`: for every valid DAG, from zero side slots, each root's slot holds the root's value.
- `word_run`: on arbitrary contents the garbage of the side slots cancels over $\mathbb{F}_2$.
- The certificate data, as Lean structures in sixteen chunks, pass `decide +kernel` in about 2.5 minutes; they give `dag_valid` and `supp_add`.
- `word_run_target`: by the triple identity $\sum_{c \in T}[c \in S,\ S \cap T = \{c\}] + \sum_{c \in T}[c \in S] \equiv [S = T] \pmod 2$, the word adds x_t into y_t for every triple and leaves everything else unchanged. `reflected_run_source` is the bank-exchanged word.

5.3. **The global node.** Stage one runs the doubled local word in invocation T with frames $K \otimes Q_T$; stage two runs it reversed and complemented, lifted to $(1 - P_S) \otimes 1 + P_S \otimes K$, with the banks exchanged. The data role X_{ST} starts in $P_S \otimes Q_T$, and after the rank-one correction Y_{ST} ends in $1 - P_S \otimes Q_T$. Since $D_U D_{1-U} = D_1$, every contract role ends with its route's stream, address fields exchanged [Lean]:

- `node_scalar`: on scalars the node maps $(x, y) \mapsto (y, x)$;
- `node_run`: the identity on streams, the field `run_identity`;
- `calls_node_le`: the node makes at most s child calls;
- `Global.pr97` : `BitNetwork`, with `runs_pr97` and `pr97_tau`.

The node uses 3217908 zero temporaries, each in the frame its first gate needs, since a zero stream is valid in every frame.

6. The machine and the seam

`M8.BitNetwork` packages a bit network: contract roles and temporaries, the width m, the prime, a control list of edges, XOR gates and resets, the route, and the certificates `calls_exact`, `legal`, `legal_mod`, `xor_gate`, `run_identity` and `branching`. `Runs` adds the shared prime, the coefficient bound and $m \le W$.

The machine `M8.Machine` is the Shared50 recursive machine made generic in the network [Lean]:

- `Induction.contract` at every depth, and `RootExecution.halts_power_bound`: the root halts within $C \cdot V \cdot w^{\tau}$ with the array transposed;
- `BinaryInterchangeRun.runs` and `BinaryInterchangeRun.uniform_bound`: the rectangular interchange in $C \cdot V \cdot \max(1, h, d)^{\tau}$, τ the network's exponent.

The arbitrary-width tail uses 43 modules of the tree unchanged and 76 ported ones.

The cost chain reaches the interchange through three modules of `Machine`: `BinaryInterchangeRun`, `BinaryInterchangeBudget` and `BinaryRadixEqualShared`. They are seams that re-export the chosen supplier, and `Parameters.tau` is the supplier's exponent; Shared50's own exponent is `Shared50Parameters.tau`. No consumer of the interchange changes. The script `scripts/s6/apply_seam.py` selects `shared50`, `m8` (Shared50 as a network on the generic machine, `Shared50Network`) or `pr97`; it is idempotent.

7. The retune

[Lean] `Global.Branching97.branching` gives $s'/W < 529^{\tau_{97}}$ for every $s' \le s$ with

$$\tau_{97} = 1 - 3734039 \cdot 10^{-12},$$

within 10^{-12} of the threshold, from a 26-term series for $\log 529$ and rationals. With $\lambda = 1 - 35 \cdot 10^{-7}$ and $\lambda' = 1 - 33 \cdot 10^{-7}$ the margins are

margin	value	slack over κ
layout $\varepsilon c(1-\tau)$	$1.582591 \cdot 10^{-6}$	$6.9 \cdot 10^{-15}$
layers $\varepsilon(1-\lambda')$	$1.598438 \cdot 10^{-6}$	$1.6 \cdot 10^{-8}$
CRT $(1-\varepsilon)(1-\tau)$	$1.925364 \cdot 10^{-6}$	$3.4 \cdot 10^{-7}$
resampling $\frac{1}{2} - \delta - \varepsilon(1 + 2^{-33})$	$7.8125 \cdot 10^{-3}$	$7.8 \cdot 10^{-3}$
direct $\varepsilon(1-c)$	$6.0547 \cdot 10^{-2}$	$6.1 \cdot 10^{-2}$

Theorem 7.1 (`MathlibTM2.mulInTime`; [Lean])**.** *One fixed `Turing.FinTM2` machine multiplies in time $O\big(n(\log n)^{1-\kappa}\big)$ with $\kappa = 111364923 \cdot 2^{-46} \approx 2^{-19.27}$.*

The F_p transform of [2] inherits the same saving through `DFT.finalHostMultiplier`.

8. Next steps

8.1. **Stage 2: the opposite-bank compiler, about $2^{-14.5}$.** The compiler of [4] replaces the r rank-one children of a frame change with one reversed child $F_{rf}(H, D) = (C_{rf}D, C_{rf}H)$ of rank rf over fields of f atoms, and stops the recursion at $w = e^{1/1000}$. On plain frames its whole-projector moment gives $a^* = 9.70 \cdot 10^{-5}$, so $\kappa \approx 2^{-14.5}$ [computation]. Done so far [Lean]:

- `exists_generic_basis_of_idempotent`: one basis with natural entries bounded in terms of m and the ranks gives every residual projector nonzero leading minors, by Mathlib's combinatorial Nullstellensatz;

- `revExchange_generic` and `revExchange_atoms`: in that basis a projector's reversed exchange is lower-triangular passes, shears and the rf interchanges of one child F_{rf}, and the adapters read only earlier atoms (`lower_kron_one`).

Remaining: the stopped recurrence $T(n,w) \le An + Bn^{\tau^*} w^{1-a_{\text{leaf}}}$ from two moment gaps; the atom adapters at cost $O(Vn)$ per node; the reversed recursive family, which needs variable-rank block children and nodes at every width; and the outer wrapper FLF. The seam stays, so the switch is again one script run and one retune.

8.2. **Further options.**

- Upstream's design-B frames raise a^* to $1.2402 \cdot 10^{-4}$, about $2^{-14.1}$, with about 15 MB of integer frame data.
- PR589 needs either one window family for all 111 ports or a two-unit corner repair; the latter would give $a_b \le 4.06 \cdot 10^{-4}$, about $2^{-12.4}$.
- The Gaussian margin limits $\varepsilon < 1/2$. The phase-cell inverse of the NearOne plan lifts it toward $2/3$, which scales every margin above by about 1.37.
- With the generic machine carrying the theorem, the Shared50 tail and root (`Shared50BinaryInterchangeRun` and its dependencies) can retire.

9. Status

item	status
Exchange lemma, exchange program, height bound, prime	Lean
Plain frames, frame heights, local word, global node	Lean
Generic machine, arbitrary-width tail, seam, Shared50 as a network	Lean
Branching certificate and retune, $\kappa = 111364923 \cdot 2^{-46}$	Lean
Shared50 family ceiling, ranking of cover-free suppliers	computation
PR589 on this machine: no certified saving	paper and computation
Stage 2: generic basis, reversed exchange over atoms	Lean
Stage 2: stopped recurrence, adapters, reversed family, wrapper	open

References

[1] D. Colkitt (maintainer) and contributors, `integer-mult-bounds`: community research on integer multiplication below $n \log n$, head `463113d6`, https://github.com/CrocSwap/integer-mult-bounds.

[2] D. Brown and K. Daniel, *The discrete Fourier transform over prime fields below $N \log N$ on multitape machines*, `docs/dft/dft.pdf` in [5].

[3] D. Brown and K. Daniel, *Hosting and realizability in the selected PR589 bit supplier*, `docs/pr589-hole/pr589-hole.pdf` in [5].

[4] icekylinx, the stopped-product notes `notes/stopped-product-*.tex` and `notes/partial-gauge-*.tex`, in [1] at `463113d6`, https://github.com/CrocSwap/integer-mult-bounds/tree/463113d675af7410ac91d53a4fed51ec6494eb9a/notes.

[5] K. Daniel, `integer-mult-bounds-lean`: integer multiplication below $n \log n$ in Lean on Mathlib's finite Turing machines, https://github.com/platypii/integer-mult-bounds-lean.

[6] OpenAI, *Integer multiplication below $n \log n$*, OpenAI Math Release preprint, September 23, 2026, https://github.com/openai/math/blob/main/preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf.

[7] Z. Chen, PR97 of [1], commit `e15350b6` of `jacklightChen/integer-mult-bounds`, https://github.com/CrocSwap/integer-mult-bounds/pull/97; pinned in [1] at `references/partial-gauge/pr97`.

[8] S. Jain, `integer-mult-kappa`, round-7 witness, commit `741e7aa0`, https://github.com/Swapnil-jain/integer-mult-kappa/tree/741e7aa078392553815df7926ee17ac5e25a8c38.

Email address: jdb1729@gmail.com

Email address: platypii@gmail.com